

End-to-End Keyword Spotting on FPGA Using Graph Neural Networks with a Neuromorphic Auditory Sensor

Wiktor Matykiewicz¹, Piotr Wzorek^{1,2}, Kamil Jeziorek^{1,2*}, Tomás Muñoz³, Antonio Rios-Navarro³, Angel Jiménez-Fernández³, and Tomasz Kryjak^{1,2}

¹ AGH University of Krakow, Poland

² Embedded Vision Systems Group, Computer Vision Laboratory

³ Robotics and Technology of Computers Lab., ETSII, EPS, SCORE, I3US, Universidad de Sevilla

* Corresponding author: kjeziorek@agh.edu.pl

Abstract. With the rapid growth of mobile robotics and embedded intelligence, there is an increasing demand for efficient on-device data processing on edge platforms. A promising research direction is the use of neuromorphic sensors inspired by human sensory systems, which generate sparse, event-based data encoding changes in the environment. In this work, we present the first end-to-end FPGA implementation of a keyword spotting system that integrates a Neuromorphic Auditory Sensor (NAS) and a graph neural network (GNN) on a single FPGA device, enabling real-time processing of raw audio data. The proposed architecture eliminates conventional signal preprocessing and operates directly on event-based audio streams. Leveraging a compute-near-memory network architecture, the system achieves efficient inference with low latency and low power consumption. Experimental results demonstrate an accuracy of 87.43% after quantization on the Google Speech Commands v2 dataset processed through the neuromorphic sensor, with end-to-end latency below 35 μ s and average power consumption of 1.12 W. The processed datasets, software models, and hardware modules are available at <https://github.com/vision-agh/NAS-GNN-KWS>.

Keywords: neuromorphic auditory sensor · FPGA · keyword spotting · graph neural network · hardware-aware design · event-based processing.

1 Introduction

Edge perception has become a key requirement in modern embedded systems, including mobile robotics and IoT applications. In such scenarios, audio processing plays an important role and must ensure high prediction accuracy while maintaining low energy consumption and minimal latency. In this context, neuromorphic sensors are gaining increasing attention, as they generate events representing changes in the environment. The resulting data are inherently sparse

in both space and time, and preserving this sparsity during processing enables a substantial reduction in computational complexity.

An example of such a device for audio processing is the Neuromorphic Auditory Sensor (NAS) [16], whose FPGA-based implementation enables direct conversion of a raw digital audio stream into event-based representations. Among the methods capable of efficiently processing sparse event data, spiking neural networks and graph neural networks (GNN) have been widely investigated. In [14], an FPGA-based architecture for keyword spotting (KWS) utilizing GNN and operating on simulated event-based data was proposed, demonstrating low latency and reduced energy consumption.

In this work, our main objective is to demonstrate the feasibility of integrating the NAS and a GNN within a single FPGA device, forming a unified end-to-end system suitable for mobile robotics applications. The main contributions of this paper are as follows:

- We present and publicly release a set of event-based audio datasets recorded using multiple NAS configurations. The datasets correspond to the popular Google Speech Commands v2 benchmark [33] for keyword spotting.
- We provide detailed statistical analysis of the recorded data and propose an event filtering method that reduces the number of processed events by approximately 47%, while simultaneously improving prediction accuracy. The method is designed for efficient hardware implementation.
- We introduce a set of architectural modifications to the GNN, derived from extensive ablation studies, enabling integration of the sensor and network on a single device while ensuring sufficient throughput. The proposed changes improve latency while reducing energy consumption.
- We present the first end-to-end KWS system operating on real event-based data from a neuromorphic auditory sensor, integrating both the sensor and a graph neural network on the same FPGA device. The system is evaluated in simulation and on the target hardware platform, achieving high accuracy, low latency, and low energy consumption.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes the proposed method. Section 4 presents the experimental evaluation. Finally, Section 5 concludes the paper.

2 Related work

2.1 Neuromorphic Auditory Sensors

Biological auditory systems operate asynchronously, generating sparse spikes only when spectral energy changes occur—unlike conventional pipelines that rely on fixed-rate sampling and computationally intensive DSP. This event-driven principle significantly reduces redundant data transmission and enables efficient processing of dynamic acoustic environments. Such bio-inspired systems offer the high dynamic range and low power consumption required for edge applications.

Early hardware realizations employed analog VLSI circuits to mimic cochlear functionality [20,6,32,18], but such systems suffer from transistor mismatch and noise, and key platforms are no longer commercially available. Digital alternatives on FPGAs offer greater scalability, though conventional filter-bank implementations retain significant arithmetic overhead [10,31]. In this work, we adopt the spike signal processing (SSP) paradigm [15] using the Neuromorphic Auditory Sensor (NAS) [16]. This architecture processes information directly in the spike domain, avoiding conventional numerical signal representations. By replacing complex arithmetic operations with simple logic gates and counters, SSP enables efficient event-based computation, allowing for fully parallel auditory processing with minimal power consumption.

2.2 Processing event-based audio data

Event-based audio from NAS is sparse, asynchronous, and temporally irregular, posing challenges for standard DSP pipelines. Learning-based methods therefore either operate directly on the event stream or convert events into temporally accumulated representations suitable for tensor-based processing.

Spiking neural networks (SNNs) remain a central modelling paradigm for event-based audio due to their native compatibility with spike-domain inputs. Existing work includes convolutional [27,11,28] and recurrent [8,24,35,9,4] architectures. More recently, state-space models (SSMs) have been investigated as an alternative framework for event-driven sequences [29,13]. SSMs provide a structured mechanism for representing evolving latent states over time and can capture long-range temporal dependencies without relying on explicit recurrence.

Another emerging direction is the use of graph neural networks (GNNs), where event streams are interpreted as collections of interacting elements rather than uniformly sampled time series [26,23]. By representing events as nodes and encoding relationships through edges, these methods can express locality and interaction patterns that are difficult to encode with fixed-grid representations. Work [14] extends this approach by integrating GNNs with recurrence to increase temporal dependency over longer contexts, which is the baseline for our study.

Alongside these algorithmic developments, a number of recent studies have focused on efficient hardware realization of event-driven processing pipelines, particularly on FPGA platforms [5,22,23,14]. Event-driven computation aligns well with FPGA architectures because sparse activity can reduce unnecessary switching and memory operations, improving energy efficiency and latency.

A recurring limitation of prior work is the reliance on synthetically generated event streams. Benchmarks such as SHD and SSC [7] are frequently used for SNN evaluation but they may not fully capture the variability, noise characteristics, and timing irregularities present in real NAS outputs. In contrast, our system processes event-based audio from a physical neuromorphic sensor integrated with a GNN on a single FPGA, alongside an efficient hardware realisation of the full processing pipeline.

3 The proposed method

The main goal of this work is to build an end-to-end keyword spotting system on a single FPGA device that processes raw audio data from a neuromorphic auditory sensor. Our system consists of three main components: the neuromorphic sensor that generates event-based audio data from raw audio input (Section 3.1), the filtration stage that reduces the number of events (Section 3.2), and the graph generation and GNN architecture that processes the event-based data (Section 3.3). In this section, we describe the implementation of each component and the overall architecture of the system.

3.1 Sensor

The input stage of our system utilizes a fully digital FPGA-based NAS architecture following the spike signal processing principles described in [15] and [16]. This module is responsible for converting continuous audio signals into a sparse, asynchronous stream of Address-Event Representation (AER) packets.

The signal acquisition pipeline begins with an analog audio input provided via a standard jack interface. An external audio codec digitizes the analog signal and transmits it to the FPGA via an I2S digital audio interface. Inside the FPGA, the pulse coded modulation (PCM) samples are immediately converted into a high-frequency spike train using pulse frequency modulation (PFM). This spike train serves as the input to a bank of digital spike-based filters that decompose the signal into distinct frequency bands (channels).

To investigate the trade-offs between biological plausibility, latency, and event throughput, we implemented and evaluated the NAS in two distinct topological configurations:

- **Cascade Architecture:** Inspired by the biological cochlea and [16], this configuration chains spike low-pass filters (SLPFs), each feeding into the next to progressively remove higher frequencies. However, cumulative latency scales with the number of channels, and non-ideal filter characteristics cause compounding signal attenuation, reducing the event rate.
- **Parallel Architecture:** Here, the input spike train feeds simultaneously into a bank of independent spike band-pass filters (SBPFs). This eliminates cumulative delay and avoids compounding signal degradation, yielding similar latency across frequency channels and higher event density (Table 1).

The output of the filter banks is transmitted via an asynchronous AER interface. Natively, the sensor encodes each event as a tuple (c, p) , representing the frequency channel index and signal polarity, respectively. To incorporate temporal information, we designed and implemented a custom timestamping module on the FPGA. This module assigns a timestamp t with a resolution of $1\mu s$ to each event immediately upon generation, yielding the final event representation (t, c, p) used for subsequent processing.

We synthesized and tested the system in six different variations to analyze the impact of spectral resolution on KWS performance. Specifically, we implemented

Table 1: Statistics of the generated neuromorphic version of the Google Speech Commands v2 dataset [34]. Reported values are the mean $\pm$ standard deviation and the maximum number of events, expressed in kilo-events (kEv.).

Version	Max. kEv./sample	Avg. kEv./sample	Max. kEv./s	Avg. kEv./s
32-cascade	204.84	17.22 ± 9.58	151.21	15.03 ± 8.21
32-parallel	227.22	39.19 ± 12.65	174.82	28.08 ± 9.12
64-cascade	227.99	31.67 ± 13.96	194.64	24.63 ± 11.15
64-parallel	245.35	75.40 ± 21.41	277.07	54.01 ± 16.14
128-cascade	254.67	61.27 ± 20.94	240.57	45.49 ± 16.01
128-parallel	265.96	126.69 ± 24.11	518.69	92.18 ± 21.03

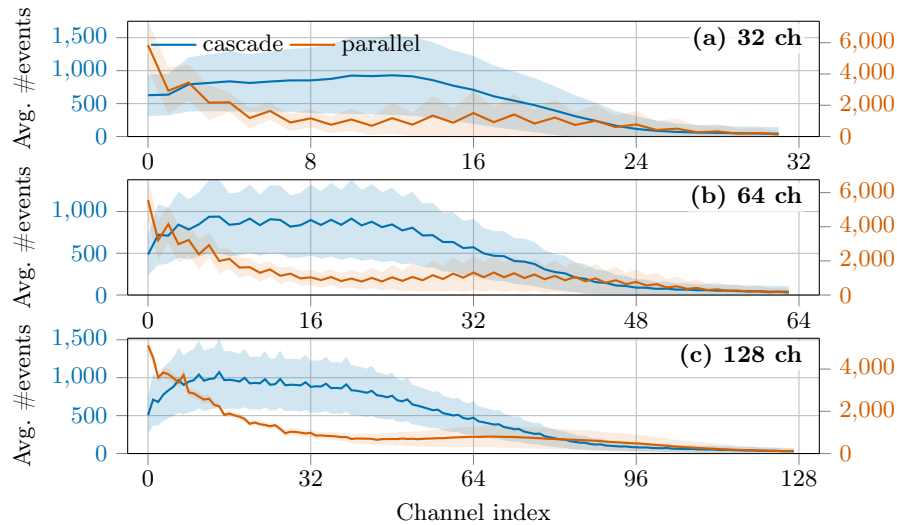


Fig. 1: Average events per channel with standard deviation for different configurations of the neuromorphic auditory sensor. For all configurations the number of events is higher for low channel (higher frequency) indices.

both cascade and parallel architectures for 32, 64, and 128 frequency channels. Increasing the number of channels improves frequency resolution but results in higher FPGA resource utilization (Slices and LUTs) and increased power consumption. The resulting event streams undergo a filtration stage before serving as the input to our graph neural network pipeline.

3.2 Filtration

As shown in Section 3.1, the sensor produces a large number of events (Table 1), and their distribution is highly uneven across channels (Fig. 1). This variability increases the computational cost of subsequent processing and can bias the graph construction toward channels with the highest activity. Therefore, before graph generation, we apply a dedicated filtration stage that reduces the event rate.

Algorithm 1: Per-channel filtration with decayed potential

Input: Event stream $\{(t_i, c_i, p_i)\}_{i=1}^N$ with time $t_i \in \mathbb{N}$, channel $c_i \in \{0, \dots, C-1\}$ and polarity $p_i \in \{-1, 1\}$
Input: Parameters: `div_factor` $\in \mathbb{N}$, weight $w \geq 0$, thresholds $\{\theta_c\}_{c=0}^{C-1}$
Output: Accepted events $\mathcal{P}$ (each as (t, c, p))
State: per-channel last timestamp $t_{\text{last}}[c]$ and potential $v[c]$
Initialization:
for $c \leftarrow 0$ **to** $C-1$ **do**
 $t_{\text{last}}[c] \leftarrow 0$
 $v[c] \leftarrow 0$
 $\mathcal{P} \leftarrow \emptyset$
 $q \leftarrow 2^{\text{div_factor}}$ // time quantization for decay
for $i \leftarrow 1$ **to** N **do**
 $t \leftarrow t_i; \quad c \leftarrow c_i; \quad p \leftarrow p_i$
 $\Delta t \leftarrow t - t_{\text{last}}[c]$
 $v[c] \leftarrow \max\left(0, v[c] - \left\lfloor \frac{\Delta t}{q} \right\rfloor\right) + w$ // decay + integrate
 $t_{\text{last}}[c] \leftarrow t$
 if $v[c] < \theta_c$ **then**
 $\mathcal{P} \leftarrow \mathcal{P} \cup \{(t, c, p)\}$ // accept event and reset potential
 $v[c] \leftarrow 0$
return $\mathcal{P}$

The proposed filtration is designed to mimic the behavior of a leaky integrate-and-fire (LIF) neuron model [1]. Each channel maintains an internal state that integrates incoming events as a discrete potential and decays over time; once the potential exceeds a channel-specific threshold, the event is accepted and the potential is reset.

In our implementation, the filtration is controlled by three parameters: `div_factor`, which defines the time quantization used for the decay; the weight w added to potential; and the per-channel thresholds θ_c . The overall procedure is summarized in Algorithm 1, which performs a per-channel update for every event and outputs a filtered event set $\mathcal{P}$.

A key element of the filtration is the selection of per-channel thresholds θ_c . To generate these thresholds, we consider three simple strategies: constant, linear, and exponential, whose effect on the event reduction and downstream performance is evaluated in the Section 4.3.

3.3 Graph Generation and GNN architecture

In our work, we adopted the FPGA-based hardware implementation proposed in [14] (for detailed description refer to full paper), designed for the keyword spotting task using event-driven audio data. The architecture follows a data-flow paradigm, where consecutive modules process the recorded event streams.

In the considered system, each input event is first forwarded to the graph generation module, where it is represented as a vertex. For every newly registered event, directed edges are created toward previously recorded events (time-directed graph) that lie within a half-sphere defined by the search radius along

the channel dimension (R_c) and the temporal dimension (R_t - we support both the lower time radius R_t^{low} and the upper time radius R_t^{high}). For this work, the values of these parameters were determined through ablation studies (see Section 4.3). This design enables sparse, event-driven processing immediately after acquisition and dynamic graph updates.

The resulting vertex, together with its edge list, is subsequently processed by the feature extraction stage composed of four consecutive **PointNetConv** modules [25]. The operation of **PointNetConv** can be expressed as:

$$\mathbf{x}'_i = \left(\max_{j \in \mathcal{N}(i) \cup \{i\}} \phi_{\Theta}(\mathbf{x}_j, \mathbf{p}_j - \mathbf{p}_i) \right), \quad (1)$$

where $\mathbf{x}_i$ denotes the input feature vector of node i , $\mathbf{p}_i$ its position, $\mathcal{N}(i)$ the neighborhood of node i , and ϕ_{Θ} is a learnable function.

For the first **PointNetConv** layer, two input features are used, corresponding to the mean neighbor position (channel and timestamp). The ablation studies were conducted to select the number of output features in each **PointNetConv** module in a way that guaranties efficient BRAM utilization, enabling full use of the memory word width.

The subsequent module is a MaxPool layer that aggregates events within 10 ms temporal windows. After this interval, the aggregated feature vector is forwarded to the network head, which consists of four linear layers and GRU-based memory unit. Every 10 ms the GNN generates prediction consisting of:

- **class**: For the keyword spotting task, we selected a set of target keywords and an additional *unknown* class that includes all remaining words, silence, and noise. The number of outputs depends on the number of selected keywords.
- **conf**: An additional single-valued output representing the confidence that a keyword has been detected within a given time window. This end-of-word temporal localization significantly improves the practical performance.

In [14], the system was evaluated on the Spiking Heidelberg Digits and Spiking Speech Commands datasets [7], which are synthetic spiking datasets obtained through simulation (with 700 audio channels each). In this work, we introduced a series of modifications to enable deployment on data recorded directly by the NAS on the same target device. To this end, we defined system-level requirements and proposed necessary optimizations.

Due to the targeted application in mobile robotics and edge processing, we identified energy consumption and system latency as the key design constraints. The proposed system must therefore exhibit low resource utilization. To further minimize energy consumption, we implemented the entire processing pipeline exclusively in reconfigurable logic, avoiding the programmable processing system of the heterogeneous platform. The raw audio signal can be acquired directly on the FPGA and processed up to prediction without CPU involvement.

At the same time, the graph neural network must ensure sufficient system throughput. To determine precise performance requirements, we analyzed the event rate after filtering (see. Section 3.3). The system must sustain real-time

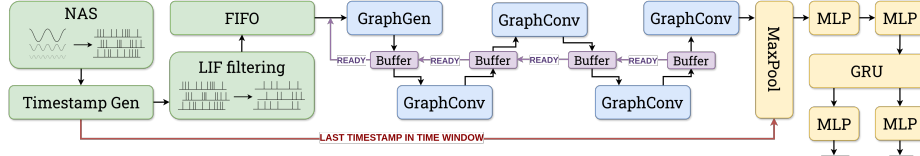


Fig. 2: The proposed architecture is illustrated with the sensor and filtering modules highlighted in green, the feature extraction stage in blue, and the MaxPool and network head modules in yellow. The scheduling mechanism is marked in purple, while the timestamp propagation mechanism is indicated in red.

processing even during periods of increased activity, which are often critical for accurate keyword spotting.

Performed modifications. The hardware architecture was designed to support all considered configurations. The NAS provides an extra attribute for each event: polarity, which we incorporated as an additional input feature to the first convolutional layer. To enable deployment on the target platform while minimizing energy consumption—without violating latency and throughput constraints—we optimized the feature extraction module of the GNN.

LUT utilization in considered architecture is dominated by vector multipliers, which are the core computational elements of the `PointNetConv` layers. While [14] employed an architecture supporting four parallel feature-vector multiplications per graph convolution (74 8-bit elements each), we reduced the number of parallel multipliers to decrease resource usage. Since graph convolution forms the system bottleneck, increasing sequential processing impacts both latency and throughput. The final number of multipliers was selected based on the measured average and maximum post-filter event rates for each NAS configuration, defining explicit requirements (see Table 3).

Moreover, the computational load of a single `PointNetConv` operation depends on the number of edges, as linear transformations must be applied to the processed vertex and each of its neighbors. In the [14] design, input events were accepted at a fixed interval (in clock cycles) determined by the worst-case neighborhood size, ensuring no internal congestion. This conservative approach resulted in constant (yet limited) throughput and unnecessary latency.

We therefore introduced a back-pressure scheduling mechanism with inter-module buffering (see Fig. 2). Each processing stage is equipped with a local buffer and signals readiness upon completing the current convolution. As the latency of each operation depends on the actual number of neighbors, events are admitted dynamically once downstream resources become available. Consequently, latency becomes event-dependent, and the input rate adapts to instantaneous workload conditions. Despite reducing the number of parallel multipliers, the effective throughput satisfies system requirements.

It is important to note that introducing the scheduling mechanism disrupts a fixed throughput. In the original design, the MaxPool module relied on a constant convolution latency, triggering feature extraction every 10 ms based on a real-time counter. After our modifications, it became necessary to implement a *timestamp propagation mechanism*, in which the NAS output timestamps are monitored and the last timestamp within each time window is propagated to the MaxPool module. The MaxPool then forwards features to the network head only when the final event of the corresponding 10 ms window has been processed.

4 Evaluation

This section evaluates the proposed keyword spotting system in terms of both classification performance and end-of-word temporal localization. We first describe the training and evaluation setup and define the metrics used throughout the experiments. We then conduct software-based ablation studies to quantify the impact of key design choices, including model capacity, graph construction, and filtration parameters, and to identify a configuration suitable for the subsequent hardware implementation.

4.1 Dataset generation

To train the proposed GNN, we utilized the Google Speech Commands v2 (GSCv2) dataset [34], comprising over 100,000 one-second utterances. To capture the precise noise characteristics and temporal dynamics of the sensor, we generated a neuromorphic version of the dataset by processing the full audio corpus through the physical NAS hardware rather than relying on software simulations.

The acquisition pipeline involved streaming GSCv2 audio samples via an I2S codec to the NAS, which decomposed the signal into asynchronous AER events. These events were captured by an Opal Kelly XEM6310 FPGA module. Utilizing the FrontPanel SDK, we established a high-bandwidth USB 3.0 communication link between the FPGA and a host PC. The FPGA logic was programmed to append microsecond-resolution timestamps to the incoming AER packets, producing (t, c, p) tuples. A dedicated Python script automated the process, synchronizing audio playback with event acquisition and converting the stream into .aedat format.

The generated datasets were approximately 15—30% longer than the original audio corpus. This extension resulted from intentional padding introduced during the hardware recording process, comprising pre-sample delays for error-free sequential acquisition, and post-sample delays to avoid positioning keywords at the end of the recording window, which degraded learning process.

4.2 Implementation details

Training Setting The keyword spotting system was trained for 50 epochs in floating point (FP32) using the Adam optimizer [17] with a learning rate of

1e-3 and weight decay of 1e-4, together with a cosine-annealing learning-rate schedule [19]. The network was then fine-tuned for 5 additional epochs using 8-bit quantization-aware training (QAT) with a constant learning rate of 1e-4. All experiments used a batch size of 16 and were run on an NVIDIA GH200 GPU. The model was implemented using PyTorch [2]. Checkpoints were selected based on the minimum validation loss and evaluated on the test set.

4.3 Ablation studies

We perform ablation studies to quantify the impact of key design choices on keyword spotting performance and to select a configuration suitable for hardware implementation. We consider three factors: model size, graph-generation settings, and filtration parameters. Unless stated otherwise, results are obtained on the test set (64-cascade configuration) using the baseline configuration: 72 channels per layer; channel radius (R_c) 20 with skip step 2; low/high time radius ($R_t^{low/high}$) of 2000/10000; and filtration parameters with division factor 8, weight 32 and thresholds exponentially spaced from 64 to 32.

We evaluate both keyword classification and end-of-word temporal localization. We report classification accuracy (Acc.) and macro F1, as well as timestamp-conditioned accuracy (Ts-acc_k), which counts a prediction as correct only if the class matches and the predicted end-of-word time falls within $\pm k$ bins (10 ms each). For the filtration ablation, we additionally report event-rate statistics. All results are in FP32 (quantized models are evaluated in Section 4.4). Formal definition of all metrics, complete sweep results and model weights are released as open-source materials⁴.

Model size We vary only the number of channels per layer in the set {18, 36, 54, 72, 90, 108, 126}. Performance improves with increasing width, reaching the best results at 126 channels per layer (79.45% accuracy, 65.51% F1). However, gains beyond 72 channels are modest: the 72-channel model achieves 78.70% accuracy and 62.29% F1, while being substantially smaller (59.84k parameters vs. 179.56k for 126 channels). We therefore select 72 channels for hardware implementation.

Graph generation We next examine the effect of graph-generation parameters.

- *Time radius.* Figure 3 shows the effect of varying the lower time radius R_t^{low} (0–5000) and upper time radius R_t^{high} (1000–10000). Increasing R_t^{low} consistently degrades all metrics: relative to $R_t^{low} = 0$, setting it to 500 or 1000 decreases the average F1 by 1.93% and 5.53%, respectively. The best results are achieved with $R_t^{low} = 0$ and $R_t^{high} \in \{2500, 5000\}$ (84.5% accuracy). We set $R_t^{low/high} = 0/5000$ for subsequent experiments.

- *Channel radius and skip step.* Table 2a reports performance for different channel-radius/skip-step pairs, with $R_t^{low/high}$ fixed to 0/5000. We consider paired settings (10/1, 20/2, 30/3, 40/4, 50/5) that keep the maximum number

⁴ <https://github.com/vision-agh/NAS-GNN-KWS>

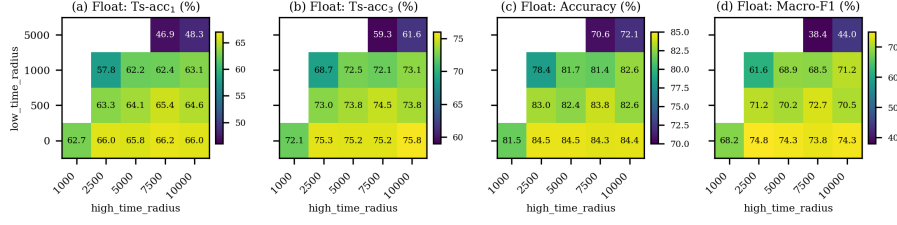


Fig. 3: Influence of the low and high time radius on the keyword-spotting metrics.

Table 2: Ablation results for (a) channel radius/skip step and (b) filtration division factor. Bold marks the selected configuration; “_” denotes no filtration.

(a) $R_t^{low/high} = 0/5000$					(b) $R_c = 10$, skip 1, $R_t^{low/high} = 0/5000$					
R_c / skip	Ts- acc ₁	Ts- acc ₃	Acc. (%)	F1 (%)	Div. fact.	Ts- acc ₁	Ts- acc ₃	Acc. (%)	F1 (%)	Avg. kEv/s
10 / 1	67.4	77.0	85.9	77.0	—	51.2	62.8	74.4	49.1	24.6
20 / 1	66.6	76.0	85.0	74.9	6	66.5	76.0	83.6	72.6	7.9
20 / 2	58.3	68.6	78.7	62.3	7	65.5	75.2	83.4	72.3	9.9
30 / 3	63.4	72.9	82.0	69.5	8	65.8	75.2	84.5	74.3	11.2
40 / 4	63.5	72.7	82.1	70.2	9	66.2	75.9	84.7	74.6	11.8
50 / 5	63.8	72.9	81.7	69.0	10	66.1	76.0	84.9	75.2	12.1

of neighbours constant (20 plus a self-loop), alongside a baseline with $R_c = 20$ and skip step 1 (dense neighbourhood). Overall, increasing either the radius or the skip step reduces performance. The best configuration is $R_c = 10$ with skip step 1 (85.89% accuracy, 76.99% F1). Together with the time-radius results, this suggests that the model benefits from a restricted spatiotemporal context, as broader neighbourhoods incorporate more irrelevant events.

Filtration We evaluate the division factor and the threshold configuration, while keeping the weight parameter constant and setting the $R_t^{low/high}$ to 0/5000.

- *Division factor.* Table 2b shows the effect of the division factor. Disabling filtration (—) substantially degrades performance (74.36% accuracy) and produces the highest event rate (24.63 avg. kEv/s). Introducing filtration improves all metrics while reducing the event count. As the division factor increases from 6 to 10, accuracy improves (83.59%→84.93%) while Ts-acc changes only marginally, indicating a trade-off between noise suppression and removal of informative events. We select division factor 8, which achieves 84.46% accuracy with an approximately twofold reduction in event rate.

- *Threshold configuration.* We compare exponential, linear, and constant threshold schedules (weight 32, division factor 8). Note that thresholds $\leq w$ effectively disable filtration (e.g. the constant-32 setting). Among all tested schedules (detailed results in the additional material), linear thresholds of 48→16 achieve the best performance (85.22% accuracy, 75.61% F1), slightly outperforming constant 48 (84.29% accuracy). This indicates that stronger filtration is beneficial

Table 3: Best ablation-selected configurations for cascade and parallel variants across datasets, reporting configuration, performance and throughput.

Dataset	Configuration						Performance (%)				Throughput		
	R_c	Skip step	R_t^{low}	R_t^{high}	div. factor	Thresholds	Ts-acc ₁	Ts-acc ₃	Acc.	F1	Max. kEv./s	Avg. kEv./s	Avg. edges/Ev
32-cascade	5	1	0	5000	8	exp.: 64 → 32	67.02	76.68	84.28	75.55	76.28	7.88	10.53
32-parallel	5	1	0	5000	8	exp.: 64 → 32	68.57	75.58	88.03	79.36	92.87	18.68	9.05
64-cascade	10	1	0	5000	8	exp.: 64 → 32	67.41	77.01	85.89	76.99	99.45	14.33	18.77
64-parallel	10	1	0	5000	8	exp.: 64 → 32	75.12	81.85	87.87	80.29	147.69	36.33	17.61
128-cascade	20	2	0	5000	8	exp.: 64 → 32	66.34	75.52	85.13	75.63	119.64	27.55	19.62
128-parallel	20	2	0	5000	8	exp.: 64 → 32	67.94	76.29	83.15	78.54	258.76	43.74	18.05

for lower channels (higher frequency), whereas higher channels (lower frequency) require less suppression.

Best configurations Based on the ablation studies, we select one configuration for each dataset variant and report the corresponding metrics in Table 3. Across all selected configurations, the time radius, division factor, and threshold settings remain identical; only the channel radius and skip step differ. The best results are achieved by the parallel configurations, which generate more than twice as many events on average as the cascade configurations. Moreover, the frequency-dependent latency introduced by the cascade configuration (see Section 3.1) hinders precise end-of-word detection, negatively affecting metrics that explicitly account for this parameter (Ts-acc₁, Ts-acc₃).

Comparison with state-of-the-art To contextualize our results, we compare the best-performing configuration identified in the ablation studies with previously reported state-of-the-art results. For consistency, we use *accuracy* as the primary metric and restrict the comparison to works reporting accuracy on the SSC dataset, which is the closest match to our dataset. In addition, we report the number of parameters as an indicator of model size (see Table 4).

Following the categorization used above, [9,4,28,11,21,3,30] report SNN-based models, [29,13] report state-space models; and [14] report a graph-based approach. For our method, we report results for the best-performing configuration within each NAS setting (32, 64, and 128 channels) and for both cascade and parallel variants, where available.

Overall, our approach achieves higher accuracy than almost all previously reported methods, including all SNN-based models and the prior graph-based implementation on SoC FPGA, while using substantially fewer parameters (59.84k). The only exception is Event-SSM [29], which reports a slightly higher peak accuracy (88.4%) than our best configuration (32-parallel, 88.03%). Importantly, our model is considerably smaller and is designed for end-to-end deployment on SoC FPGA, making it well suited for full hardware implementation under tight resource constraints.

Table 4: Accuracy and parameter-count comparison. Results for prior work are reported on the SSC dataset, while our models are evaluated on our dataset. Ranges indicate results reported for multiple model settings in the cited work.

Category	Reference	Model	#Params	Acc. (%)
<i>SNN</i>	Dampfhofer et al. [9]	SpikGRU	280k	77.0
	Bittar et al. [4]	Recurrent SNN	3.9M	77.4
	Sadovsky et al. [28]	SNN-CNN	–	72.0
	Hammouamri et al. [11]	DCLS-Delays	0.7–2.5M	79.8–80.7
	Malettira et al. [21]	Temporal skips with delay learning	1.4M	80.2
	Baronig et al. [3]	SE-adLIF	1.6M	80.4
	Sun et al. [30]	PfA SNN	0.1/0.7M	77.4/80.2
<i>SSM</i>	Schone et al. [29]	Event-State-Space Model	0.1/0.6M	85.3/88.4
	Huber et al. [13]	S5-RF	1.8M	78.8
<i>GNN</i>	Jeziorek et al. [14]	Spectro-temporal graph on SoC FPGA	8.6k–272k	78.4–84.3
<i>Our</i>	32-cascade	End-to-end GNN with NAS on FPGA	59.84k	84.61
	32-parallel		59.84k	88.03
	64-cascade		59.84k	85.89
	64-parallel		59.84k	87.87
	128-cascade		59.84k	85.03
	128-parallel		59.84k	83.15

4.4 Hardware implementation

Based on the ablation studies (Section 4.3) and the system requirements (Section 3.3), we developed a hardware implementation integrating both the NAS and the GNN on the same device, building upon the modules provided in [14].

The graph neural network module supports all considered NAS configurations (32, 64, or 128 channels, in both parallel and serial variants). Due to efficient memory organization for storing features of previously processed events—required for the `PointNetConv` operation—BRAM usage remains independent of the channel count. Each memory entry stores nine 8-bit features (72 bits in total), resulting in a required depth of 1024 for the 128-channel configuration.

Based on measured event statistics, we selected the number of parallel multipliers per graph convolution to process two 72-element feature vectors concurrently. With this configuration, a new event can be accepted every 0.47 μ s to 4.07 μ s (depending on the number of edges) at 200 MHz. The resulting throughput is estimated between 2.1 MEv./s and 245 kEv./s, significantly exceeding the measured event rates for all considered datasets (see Table 3). To handle short bursts of higher activity, we introduced a FIFO buffer between the filtering and graph generation modules.

The complete system was implemented on AMD US+ ZCU104 and validated against the software reference model. For evaluation purposes, we selected the 32-channel and 64-channel parallel NAS configurations due to their high accuracy of 87.43% and 86.9% respectively (after 8-bit quantization). An additional advantage of the parallel variants is the elimination of additional low-frequency latency effects that are characteristic of the cascaded configuration. Both implementations were evaluated with respect to the following metrics (Table 5):

Table 5: Evaluation of proposed design compared with GNN from [14].

Metric	GNN [14]	64-parallel This work	32-parallel This work
LUT	125,130	88,370	82,739
FF	82,372	84,031	77,457
BRAM	75.5	55.5	55.5
DSP	140	83	83
Latency [μ s]	10.53	25 (42)	25 (35)
Throughput [keps]	555	245	440
Power [W]	1.18	1.16	1.12
Accuracy	73.5%	86.9%	87.43%

- **Resource utilization** The proposed optimizations enabled a significant reduction in resource utilization compared to the GNN proposed in [14]. This resource efficiency allowed the full system to be deployed on the target platform without timing violations at a 200 MHz clock frequency.
- **Latency.** Due to the MaxPool architecture, the system generates predictions every 10 ms, invoking the network head once per window. With the timestamp propagation mechanism (see Section 3.3), the effective GNN latency—defined as the time between the end of a 10 ms window and the prediction—is typically equal to the head latency, i.e., 2.11 μ s. In the worst case (when the last event occurs near the very end of the time window), it increases to a maximum of 18.62 μ s. The NAS exhibits an input-to-event latency of 23 μ s resulting in overall end-to-end latency between 25 and 42 μ s (for 32-channels between 25 and 35 μ s).
- **Power consumption.** Average power of the KWS modules was estimated in Vivado using post-implementation simulation toggle-rates under 40 kEv/s load. For 64-channel configuration (simulated with an average of 18 edges/Ev.) yielding 0.539 W dynamic and 0.595 W static, and for 32-channels (avg. 9.8 edges/Ev.) - 0.494 W dynamic and 0.595 W static. With NAS power reported as 29.7 mW in [16], the total average power is estimated at 1.16 W and 1.12 W, for 64-channel and 32-channel configurations respectively.

To the best of our knowledge, this is the first system enabling fully integrated, end-to-end keyword spotting on event-based audio with both the sensor and neural network implemented on a single FPGA device. There are, however, recent works on efficient KWS implementations for FPGA platforms evaluated on the original Google Speech Commands Dataset (not the spiking one) that report higher accuracies, e.g., a binary neural network accelerator on a Xilinx VC707 achieving 97.29% in 536 clock cycles [36], and a configurable temporal-efficient neural network reaching 95.36% [12]. However, both approaches rely on precomputed features rather than processing raw audio directly, which introduces additional preprocessing overhead, power consumption, and latency. Furthermore, their predictions are generated once per 1 s input sample.

In contrast, our approach eliminates the need for explicit aggregation or conventional preprocessing stages, thereby avoiding the associated latency and computational overhead. Furthermore, we generate predictions every 10 ms, significantly increasing the temporal resolution of keyword detection. This combination of raw audio handling, frequent prediction updates, and adaptive scheduling distinguishes our architecture from existing FPGA-based KWS systems.

5 Summary

This paper presents an end-to-end, real-time keyword spotting system implemented entirely on a single FPGA through tight integration of a Neuromorphic Auditory Sensor (NAS) with a graph neural network (GNN). We publicly release both software and hardware implementations, together with recorded event-based versions of the GSCD for multiple NAS configurations.

We propose a hardware-friendly event filtering method that reduces the number of processed events by approximately 47% while improving classification accuracy. Furthermore, we introduce GNN architectural modifications derived from extensive ablation studies to reduce resource utilization, latency, and power consumption while maintaining high throughput. On hardware, the selected 64-channel parallel NAS configuration achieves 87.43% accuracy after 8-bit quantization. The system produces predictions every 10 ms, with an effective post-window inference latency of 35 μ s and low average power consumption at 1.12W.

The proposed design enables accurate and high-speed inference directly from raw audio on a single device, making it well suited for edge computing applications in mobile robotics and IoT. In future work, we plan to validate the proposed approach in real-world robotic human-machine interaction scenarios. Additionally, we intend to investigate the feasibility of an ASIC implementation to achieve an even more efficient and optimized solution.

6 Acknowledgments

This work was supported by: the Polish National Science Centre projects no. 2024/53/N/ST6/04254 and no. 2024/53/N/ST6/04331, the “Excellence initiative – research university” programme, 16.16.120.773 for the AGH University of Krakow. This work is a part of the projects PID2023-149071NB-C54 (NEKOR) funded by “Ministerio de Ciencia, Innovación y Universidades”/AEI/10.13039/501100011033, by “ERDF A way of making Europe” and by the European Union NextGenerationEU/PRTR, by grant USECHIP (TSI-069100-2023-001), project funded by the Secretary of State for Telecommunications and Digital Infrastructure, Ministry for Digital Transformation and Civil Service and by the European Union-NextGenerationEU/PRTR. We gratefully acknowledge Polish high-performance computing infrastructure PLGrid (HPC Center: ACK Cyfronet AGH) for providing computer facilities and support within computational grant no. PLG/2026/019156.

References

1. Abbott, L.F.: Lapicque's introduction of the integrate-and-fire model neuron (1907). *Brain research bulletin* **50**(5-6), 303–304 (1999)
2. Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., Hirsh, B., Huang, S., Kalambarkar, K., Kirsch, L., Lazos, M., Lezcano, M., Liang, Y., Liang, J., Lu, Y., Luk, C., Maher, B., Pan, Y., Puhersch, C., Reso, M., Saroufim, M., Siraichi, M.Y., Suk, H., Suo, M., Tillet, P., Wang, E., Wang, X., Wen, W., Zhang, S., Zhao, X., Zhou, K., Zou, R., Mathews, A., Chanan, G., Wu, P., Chintala, S.: PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In: 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24). ACM (Apr 2024). <https://doi.org/10.1145/3620665.3640366>, <https://docs.pytorch.org/assets/pytorch2-2.pdf>
3. Baronig, M., Ferrand, R., Sabathiel, S., Legenstein, R.: Advancing spatio-temporal processing through adaptation in spiking neural networks. *Nature Communications* **16**(1), 5776 (Jul 2025). <https://doi.org/10.1038/s41467-025-60878-z>, <https://doi.org/10.1038/s41467-025-60878-z>
4. Bittar, A., Garner, P.N.: A surrogate gradient spiking baseline for speech command recognition. *Frontiers in Neuroscience* **16** (2022). <https://doi.org/10.3389/fnins.2022.865897>
5. Carpegna, A., Savino, A., Carlo, S.D.: Spiker+: a framework for the generation of efficient Spiking Neural Networks FPGA accelerators for inference at the edge. *IEEE Transactions on Emerging Topics in Computing* (01), 1–15 (Dec 2024). <https://doi.org/10.1109/TETC.2024.3511676>
6. Chan, V., Liu, S.C., van Schaik, A.: Aer ear: A matched silicon cochlea pair with address event representation interface. *IEEE Transactions on Circuits and Systems I: Regular Papers* **54**(1), 48–59 (2007). <https://doi.org/10.1109/TCSI.2006.887979>
7. Cramer, B., Stradmann, Y., Schemmel, J., Zenke, F.: The Heidelberg Spiking Data Sets for the Systematic Evaluation of Spiking Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems* pp. 1–14 (2020). <https://doi.org/10.1109/TNNLS.2020.3044364>
8. Cramer, B., Stradmann, Y., Schemmel, J., Zenke, F.: The heidelberg spiking data sets for the systematic evaluation of spiking neural networks. *IEEE Transactions on Neural Networks and Learning Systems* **33**(7), 2744–2757 (2022). <https://doi.org/10.1109/TNNLS.2020.3044364>, <https://zenkelab.org/resources/spiking-heidelberg-datasets-shd/>
9. Dampfhofer, M., Mesquida, T., Valentian, A., Anghel, L.: Investigating current-based and gating approaches for accurate and energy-efficient spiking recurrent neural networks. In: Pimenidis, E., Angelov, P., Jayne, C., Papaleonidas, A., Aydin, M. (eds.) *Artificial Neural Networks and Machine Learning – ICANN 2022*. pp. 359–370. Springer Nature Switzerland, Cham (2022). https://doi.org/10.1007/978-3-031-15934-3_30
10. Gambin, I., Grech, I., Casha, O., Gatt, E., Micallef, J.: Digital cochlea model implementation using xilinx xc3s500e spartan-3e fpga. In: 2010 17th IEEE International Conference on Electronics, Circuits and Systems. pp. 946–949 (2010). <https://doi.org/10.1109/ICECS.2010.5724669>

11. Hammouamri, I., Khalfaoui-Hassani, I., Masquelier, T.: Learning delays in spiking neural networks using dilated convolutions with learnable spacings (2023), <https://arxiv.org/abs/2306.17670>
12. He, K., Chen, D., Su, T.: A configurable accelerator for keyword spotting based on small-footprint temporal efficient neural network. *Electronics* **11**(16), 2571 (2022)
13. Huber, T.E., Lecomte, J., Polovnikov, B., von Arnim, A.: Scaling up resonate-and-fire networks for fast deep learning. In: Del Bue, A., Canton, C., Pont-Tuset, J., Tommasi, T. (eds.) *Computer Vision – ECCV 2024 Workshops*. pp. 241–258. Springer Nature Switzerland, Cham (2025). https://doi.org/10.1007/978-3-031-92460-6_15
14. Jeziorek, K., Wzorek, P., Blachut, K., Nakano, H., Dampfhofer, M., Mesquida, T., Nishi, H., Dalgaty, T., Kryjak, T.: Hardware-accelerated graph neural networks: an alternative approach for neuromorphic event-based audio classification and keyword spotting on soc fpga (2026), <https://arxiv.org/abs/2602.16442>
15. Jimenez-Fernandez, A., Linares-Barranco, A., Paz-Vicente, R., Jiménez, G., Civit, A.: Building blocks for spikes signals processing. In: *The 2010 International Joint Conference on Neural Networks (IJCNN)*. pp. 1–8 (2010). <https://doi.org/10.1109/IJCNN.2010.5596845>
16. Jiménez-Fernández, A., Cerezuela-Escudero, E., Miró-Amarante, L., Domínguez-Morales, M.J., Gomez-Rodriguez, F., Linares-Barranco, A., Jiménez-Moreno, G.: A binaural neuromorphic auditory sensor for fpga: A spike signal processing approach. *IEEE Trans. Neural Netw. Learning Syst.* **28**(4), 804–818 (2017)
17. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014)
18. Liu, S.C., van Schaik, A., Minch, B.A., Delbruck, T.: Asynchronous binaural spatial audition sensor with $2 \times 64 \times 4$ channel output. *IEEE Transactions on Biomedical Circuits and Systems* **8**(4), 453–464 (2014). <https://doi.org/10.1109/TBCAS.2013.2281834>
19. Loshchilov, I., Hutter, F.: Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983* (2016)
20. Lyon, R., Mead, C.: An analog electronic cochlea. *IEEE Transactions on Acoustics, Speech, and Signal Processing* **36**(7), 1119–1134 (1988). <https://doi.org/10.1109/29.1639>
21. Malettira, P.G., Negi, S., Ponghiran, W., Roy, K.: TSkip: Efficiency Through Explicit Temporal Delay Connections in Spiking Neural Networks (2024), <https://arxiv.org/abs/2411.16711>
22. Matinizadeh, S., Pacik-Nelson, N., Polykretis, I., Tishbi, K., Kumar, S., Varshika, M.L., Mohammadhassani, A., Mishra, A., Kandasamy, N., Shackleford, J., Gallo, E., Das, A.: A fully-configurable open-source software-defined digital quantized spiking neural core architecture (2024), <https://arxiv.org/abs/2404.02248>
23. Nakano, H., Blachut, K., Jeziorek, K., Wzorek, P., Dampfhofer, M., Mesquida, T., Nishi, H., Kryjak, T., Dalgaty, T.: Hardware-accelerated event-graph neural networks for low-latency time-series classification on soc fpga. In: *International Symposium on Applied Reconfigurable Computing*. pp. 51–68. Springer (2025)
24. Perez-Nieves, N., Leung, V.C.H., Dragotti, P.L., Goodman, D.F.M.: Neural heterogeneity promotes robust learning. *Nature Communications* **12**(1), 5791 (Oct 2021). <https://doi.org/10.1038/s41467-021-26022-3>, <https://doi.org/10.1038/s41467-021-26022-3>
25. Qi, C.R., Yi, L., Su, H., Guibas, L.J.: Pointnet++: Deep hierarchical feature learning on point sets in a metric space (2017), <https://arxiv.org/abs/1706.02413>

26. Rafeldt, L., Mesquida, T., Nakano, H., Dampfhofer, M., Moro, F., Vivet, P., Payvand, M., Dalgaty, T.: Event-based audio prediction with spectro-temporal event-graphs. In: 2025 IEEE International Symposium on Circuits and Systems (ISCAS). pp. 1–5 (2025). <https://doi.org/10.1109/ISCAS56072.2025.11043865>
27. Rossbroich, J., Gygax, J., Zenke, F.: Fluctuation-driven initialization for spiking neural network training. *Neuromorphic Computing and Engineering* **2**(4), 044016 (Dec 2022). <https://doi.org/10.1088/2634-4386/ac97bb>
28. Sadosky, E., Jakubec, M., Jarina, R.: Speech command recognition based on convolutional spiking neural networks. In: 2023 33rd International Conference Radioelektronika (RADIOELEKTRONIKA). pp. 1–5 (2023). <https://doi.org/10.1109/RADIOELEKTRONIKA57919.2023.10109082>
29. Schöne, M., Sushma, N.M., Zhuge, J., Mayr, C., Subramoney, A., Kappel, D.: Scalable event-by-event processing of neuromorphic sensory signals with deep state-space models. In: 2024 International Conference on Neuromorphic Systems (ICONS). pp. 124–131 (2024). <https://doi.org/10.1109/ICONS62911.2024.00026>
30. Sun, P., Wu, J., Devos, P., Botteldooren, D.: Towards parameter-free attentional spiking neural networks. *Neural Networks* **185**, 107154 (2025). <https://doi.org/https://doi.org/10.1016/j.neunet.2025.107154>, <https://www.sciencedirect.com/science/article/pii/S0893608025000334>
31. Thakur, C.S., Hamilton, T.J., Tapson, J., van Schaik, A., Lyon, R.F.: Fpga implementation of the car model of the cochlea. In: 2014 IEEE International Symposium on Circuits and Systems (ISCAS). pp. 1853–1856 (2014). <https://doi.org/10.1109/ISCAS.2014.6865519>
32. Wang, S., Koickal, T.J., Enemali, G., Gouveia, L., Wang, L., Hamilton, A.: Design of a silicon cochlea system with biologically faithful response. In: 2015 International Joint Conference on Neural Networks (IJCNN). pp. 1–7 (2015). <https://doi.org/10.1109/IJCNN.2015.7280828>
33. Warden, P.: Speech commands: A dataset for limited-vocabulary speech recognition. arXiv preprint arXiv:1804.03209 (2018)
34. Warden, P.: Speech commands: A dataset for limited-vocabulary speech recognition (2018), <https://arxiv.org/abs/1804.03209>
35. Yin, B., Corradi, F., Bohtë, S.M.: Accurate and efficient time-domain classification with adaptive spiking recurrent neural networks. *Nature Machine Intelligence* **3**(10), 905–913 (Oct 2021). <https://doi.org/10.1038/s42256-021-00397-w>, <https://doi.org/10.1038/s42256-021-00397-w>
36. Zhang, A., Shi, J., Qian, H., Wang, J.: High precision speech keyword spotting based on binary deep neural network in fpga. *Entropy* **27**(11), 1143 (2025)